



Database Management System

IT214

Final Project Submission

Supply-Chain Management System (*SCMS*)

Lab Group: 5

Name of Student	Student ID
-----------------	------------

Darshan Talati	202401046
----------------	-----------

Kavya Bhojwani	202401090
----------------	-----------

Shlok Patel	202401156
-------------	-----------

Aaryan Modi	202401435
-------------	-----------

Group Representative: Darshan Talati (202401046) (M. 7861024620)

Acknowledgements

We would like to express our sincere gratitude to **Dhirubhai Ambani University** (formerly DA-IICT) for providing the academic framework within which this project was undertaken.

We are deeply thankful to our **IT214 — Database Management System** course instructor **Prof. PM Jat**, laboratory faculty and teaching assistants, especially **Kahan Mehta** and **Talaksh Rabari**, for their continuous guidance, timely feedback across all milestones, and for designing a project structure that encouraged authentic, industry-adjacent database design.

This project — the *Supply Chain Management System (SCMS)* — was developed collaboratively by the authors of this document. Each team member contributed meaningfully to every section as and when required. The iterative milestone structure provided valuable experience in evolving a schema under real design constraints.

We also acknowledge the open-access literature on relational database theory — in particular the foundational works of E.F. Codd and Ramez Elmasri — which informed our normalization methodology and FD analysis.

Finally, we thank our peers for healthy academic discussion and cross-group reviews that sharpened several design decisions documented herewith.

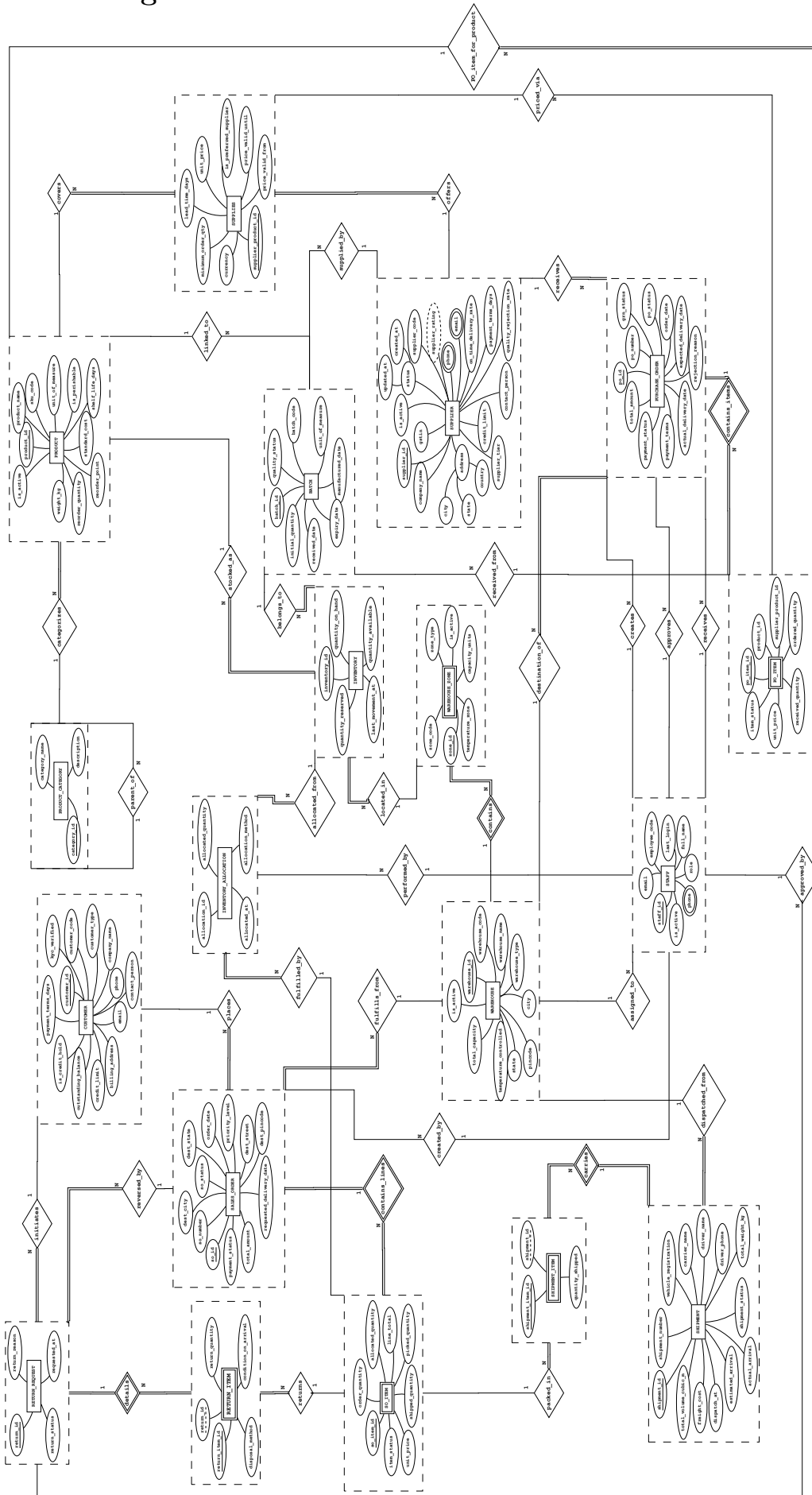
Darshan Talati	202401046
Kavya Bhojwani	202401090
Shlok Patel	202401156
Aaryan Modi	202401435

IT214 — Database Management System
Dhirubhai Ambani University (formerly DA-IICT), Gandhinagar

Contents

Acknowledgements	2
1 ER Diagram	4
2 Relational Schema	5
3 Minimal Functional Dependency Sets	6
4 Normalization Proofs (1NF $\rightarrow$ BCNF), Table-Wise	15
5 Notes	24

1 ER Diagram



3 Minimal Functional Dependency Sets

The following tables present the minimal (canonical) FD set for each relation. Trivial FDs are omitted. For surrogate-PK tables, the natural candidate key FD is always listed separately.

SUPPLIER (Strong Entity)

Two candidate keys exist beyond the surrogate PK: `supplier_code` (business identifier), `gstn` (tax registration), and `email` — each unique by constraint. The generated column `supplier_rating` is determined by two non-key columns.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
<code>supplier_id</code>	→	{ <code>supplier_code</code> , <code>company_name</code> , <code>contact_person</code> , <code>email</code> , <code>phone</code> , <code>gstn</code> , <code>city</code> , <code>state</code> , <code>country</code> , <code>credit_limit</code> , <code>payment_terms_days</code> , <code>supplier_tier</code> , <code>on_time_delivery_rate</code> , <code>quality_rejection_rate</code> , <code>is_active</code> , <code>created_at</code> , <code>updated_at</code> }	PK — single determinant for all non-key attributes
<code>supplier_code</code>	→	{ <code>supplier_id</code> } (+ all other attributes via <code>supplier_id</code>)	Alternate CK — UNIQUE constraint
<code>gstn</code>	→	{ <code>supplier_id</code> } (+ all other attributes via <code>supplier_id</code>)	Alternate CK — UNIQUE constraint
<code>email</code>	→	{ <code>supplier_id</code> } (+ all other attributes via <code>supplier_id</code>)	Alternate CK — UNIQUE constraint
{ <code>on_time_delivery_rate</code> , <code>quality_rejection_rate</code> }	→	<code>supplier_rating</code>	GENERATED ALWAYS AS STORED: $\text{ROUND}((\text{on_time} \times 0.6) + ((100 - \text{rejection}) \times 0.4), 2)$. LHS is not a superkey — documented BCNF violation.

PRODUCT_CATEGORY (Strong Entity — Recursive)

Two candidate keys: `category_id` (surrogate PK) and `category_name` (UNIQUE). The self-referential `parent_category_id` FK is an attribute, not a source of intra-table FDs beyond the PK.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
<code>category_id</code>	→	{ <code>category_name</code> , <code>parent_category_id</code> , <code>description</code> }	PK
<code>category_name</code>	→	{ <code>category_id</code> , <code>parent_category_id</code> , <code>description</code> }	Alternate CK — UNIQUE constraint

PRODUCT (Strong Entity)

Two candidate keys: `product_id` (surrogate PK) and `sku_code` (UNIQUE). All product attributes describe a single product entity directly.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
product_id	→	{sku_code, product_name, category_id, unit_of_measure, is_perishable, shelf_life_days, standard_cost, reorder_point, reorder_quantity, weight_kg, is_active}	PK
sku_code	→	{product_id, product_name, category_id, unit_of_measure, is_perishable, shelf_life_days, standard_cost, reorder_point, reorder_quantity, weight_kg, is_active}	Alternate CK — UNIQUE constraint

SUPPLIES (Associative Entity — M:N (Supplier × Product))

Surrogate PK supplier_product_id and natural candidate key {supplier_id, product_id, price_valid_from}. No partial dependency can exist because all non-key attributes describe the pricing contract for a specific supplier-product-period triple.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
supplier_product_id	→	{supplier_id, product_id, unit_price, currency, minimum_order_qty, lead_time_days, price_valid_from, price_valid_until, is_preferred_supplier}	Surrogate PK
{supplier_id, product_id, price_valid_from}	→	{supplier_product_id, unit_price, currency, minimum_order_qty, lead_time_days, price_valid_until, is_preferred_supplier}	Natural CK — UNIQUE constraint on triple

WAREHOUSE (Strong Entity)

Two candidate keys: warehouse_id (surrogate PK) and warehouse_code (UNIQUE). All attributes describe warehouse properties directly.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
warehouse_id	→	{warehouse_code, warehouse_name, warehouse_type, city, state, pincode, total_capacity_sqft, temperature_controlled, is_active}	PK

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
warehouse_code	→	{warehouse_id, warehouse_name, warehouse_type, city, state, pincode, total_capacity_sqft, temperature_controlled, is_active}	Alternate CK — UNIQUE constraint

WAREHOUSE_ZONE (Weak Entity — owner: Warehouse)

Surrogate PK zone_id and composite natural CK {warehouse_id, zone_code}. zone_code alone is only a partial key (unique within a warehouse, not globally). All non-key attributes describe zone properties.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
zone_id	→	{warehouse_id, zone_code, zone_type, capacity_units, temperature_zone, is_active}	Surrogate PK
{warehouse_id, zone_code}	→	{zone_id, zone_type, capacity_units, temperature_zone, is_active}	Natural composite CK — UNIQUE(warehouse_id, zone_code)

BATCH (Strong Entity)

Two candidate keys: batch_id (surrogate PK) and batch_code (UNIQUE). All attributes describe a specific physical lot received from a supplier.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
batch_id	→	{batch_code, product_id, supplier_id, po_item_id, manufactured_date, expiry_date, received_date, initial_quantity, unit_of_measure, quality_status}	PK
batch_code	→	{batch_id, product_id, supplier_id, po_item_id, manufactured_date, expiry_date, received_date, initial_quantity, unit_of_measure, quality_status}	Alternate CK — UNIQUE constraint

INVENTORY (Ternary Associative Entity — Product × Batch × Warehouse_Zone)

Surrogate PK inventory_id; natural CK {batch_id, zone_id}. Two additional FDs arise from design choices: batch_id → product_id (cross-table transitive redundancy) and the generated quantity_available.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
inventory_id	→	{product_id, batch_id, zone_id, quantity_on_hand, quantity_reserved, last_movement_at}	Surrogate PK
{batch_id, zone_id}	→	{inventory_id, product_id, quantity_on_hand, quantity_reserved, last_movement_at}	Natural CK — UNIQUE(batch_id, zone_id)
batch_id	→	product_id	Cross-table transitive FD: Batch.product_id is already determined by batch_id in the BATCH table. Storing product_id here introduces redundancy — documented BCNF violation, retained for join-free query performance.
{quantity_on_hand, quantity_reserved}	→	quantity_available	GENERATED ALWAYS AS STORED: quantity_on_hand – quantity_reserved. LHS is not a superkey — additional normalization note.

INVENTORY_ALLOCATION (Associative Entity — SO_Item × Inventory)

Single surrogate PK allocation_id. No composite natural key enforced — each allocation event is a unique time-stamped record.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
allocation_id	→	{so_item_id, inventory_id, allocated_quantity, allocation_method, allocated_at, allocated_by}	PK — only determinant in this table

PURCHASE_ORDER (Strong Entity)

Two candidate keys: po_id (surrogate PK) and po_number (UNIQUE business identifier). GRN fields are merged in from the eliminated GRN table; they are all fully determined by po_id (one GRN per PO, 1:1 relationship).

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
po_id	→	{po_number, supplier_id, warehouse_id, created_by, approved_by, po_status, order_date, expected_delivery_date, actual_delivery_date, received_by, grn_status, rejection_reason, payment_terms, payment_status, total_amount}	PK
po_number	→	{po_id} (+ all attributes via po_id)	Alternate CK — UNIQUE constraint

PO_ITEM (Weak Entity — owner: Purchase_Order)

Surrogate PK po_item_id; natural CK {po_id, product_id}. The stored line_total is computed from ordered_quantity × unit_price — introducing a functional dependency where the LHS is not a superkey.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
po_item_id	→	{po_id, product_id, supplier_product_id, ordered_quantity, received_quantity, unit_price, line_total, item_status}	Surrogate PK
{po_id, product_id}	→	{po_item_id, supplier_product_id, ordered_quantity, received_quantity, unit_price, line_total, item_status}	Natural CK — UNIQUE(po_id, product_id)
{ordered_quantity, unit_price}	→	line_total	GENERATED ALWAYS AS STORED: ordered_quantity × unit_price. This is an audit snapshot. LHS (ordered_quantity, unit_price) is not a superkey — BCNF violation, retained as price-at-order-time snapshot.

CUSTOMER (Strong Entity)

Three candidate keys: customer_id (PK), customer_code (UNIQUE), and email (UNIQUE). outstanding_balance is NOT stored in the table — it is derived at query time via SUM over Sales_Order.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
customer_id	→	{customer_code, customer_type, company_name, contact_person, email, phone, billing_address, credit_limit, is_credit_hold, payment_terms_days, kyc_verified}	PK
customer_code	→	{customer_id} (+ all attributes via customer_id)	Alternate CK — UNIQUE constraint
email	→	{customer_id} (+ all attributes via customer_id)	Alternate CK — UNIQUE constraint

SALES_ORDER (Strong Entity)

Two candidate keys: so_id (surrogate PK) and so_number (UNIQUE). Destination address is stored as decomposed flat fields.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
so_id	→	{so_number, customer_id, warehouse_id, created_by, so_status, order_date, requested_delivery_date, priority_level, dest_street, dest_city, dest_state, total_amount, payment_status}	PK
so_number	→	{so_id} (+ all attributes via so_id)	Alternate CK — UNIQUE constraint

SO_ITEM (Weak Entity — owner: Sales_Order)

Surrogate PK so_item_id; natural CK {so_id, product_id}. Parallel structure to PO_ITEM: line_total is stored as an audit snapshot introducing the same BCNF consideration.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
so_item_id	→	{so_id, product_id, ordered_quantity, allocated_quantity, picked_quantity, shipped_quantity, unit_price, line_total, item_status}	Surrogate PK
{so_id, product_id}	→	{so_item_id, ordered_quantity, allocated_quantity, picked_quantity, shipped_quantity, unit_price, line_total, item_status}	Natural CK — UNIQUE(so_id, product_id)

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
{ordered_quantity, unit_price}	→	line_total	GENERATED ALWAYS AS STORED: ordered_quantity × unit_price. Audit snapshot — same analysis as PO_ITEM. BCNF violation retained with justification.

SHIPMENT (Strong Entity)

Two candidate keys: shipment_id (surrogate PK) and shipment_number (UNIQUE). A notable intra-table transitive FD exists: driver_name → driver_phone, which is a 3NF violation.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
shipment_id	→	{shipment_number, origin_warehouse_id, vehicle_registration, carrier_name, driver_name, driver_phone, shipment_status, dispatch_at, estimated_arrival, actual_arrival, total_weight_kg, total_volume_cubic_m, freight_cost}	PK
shipment_number	→	{shipment_id} (+ all attributes via shipment_id)	Alternate CK — UNIQUE constraint
driver_name	→	driver_phone	Intra-table transitive FD: driver_name is not a superkey of SHIPMENT. This is a genuine 3NF violation — driver_phone is a non-prime attribute determined by a non-prime attribute. Fix: extract DRIVER table. Retained to avoid a 20th table.

SHIPMENT_ITEM (Weak Entity — owner: Shipment)

Surrogate PK shipment_item_id; natural CK {shipment_id, so_item_id}. Minimal table with only three payload columns.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
shipment_item_id	→	{shipment_id, so_item_id, quantity_shipped}	Surrogate PK
{shipment_id, so_item_id}	→	{shipment_item_id, quantity_shipped}	Natural CK — UNIQUE(shipment_id, so_item_id)

RETURN_REQUEST (Strong Entity)

Surrogate PK return_id. Contains a deliberate denormalization: customer_id is stored directly even though it is transitively determined via return_id → so_id → customer_id. This is a genuine 3NF

(and BCNF) violation.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
return_id	→	{so_id, customer_id, approved_by, return_reason, return_status, requested_at}	PK
so_id	→	customer_id	Cross-table transitive FD: so_id → customer_id holds because Sales_Order.customer_id is determined by so_id. Storing customer_id here creates a transitive dependency within RETURN_REQUEST — 3NF violation, retained as deliberate denormalization for direct customer-level query access.

RETURN_ITEM (Weak Entity — owner: Return_Request)

Surrogate PK return_item_id; natural CK {return_id, so_item_id}. restocked_to_inventory_id is a nullable FK — it does not introduce intra-table FDs beyond the PK.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
return_item_id	→	{return_id, so_item_id, returned_quantity, condition_on_arrival, restocked_to_inventory_id, disposal_method}	Surrogate PK
{return_id, so_item_id}	→	{return_item_id, returned_quantity, condition_on_arrival, restocked_to_inventory_id, disposal_method}	Natural CK — UNIQUE(return_id, so_item_id)

STAFF (Strong Entity)

Three candidate keys: staff_id (PK), employee_code (UNIQUE), and email (UNIQUE). warehouse_id is a nullable FK — its nullability indicates central staff and does not create an intra-table transitive dependency.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
staff_id	→	{employee_code, full_name, role, email, phone, warehouse_id, is_active, last_login}	PK
employee_code	→	{staff_id} (+ all attributes via staff_id)	Alternate CK — UNIQUE constraint
email	→	{staff_id} (+ all attributes via staff_id)	Alternate CK — UNIQUE constraint

REORDER_ALERT (System-Managed (Trigger-Populated))

Trigger-populated table. Single surrogate PK alert_id. reorder_point is a snapshot value captured at the time of alert generation.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
alert_id	→	{product_id, warehouse_id, preferred_supplier_id, current_stock, reorder_point, suggested_order_quantity, alert_status, triggered_at}	PK — only determinant

AUDIT_LOG (Append-Only Log (Trigger-Managed))

Immutable audit log. Single BIGSERIAL PK log_id. old_value and new_value are JSONB blobs — opaque atomic values, not repeating groups.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
log_id	→	{staff_id, action_type, table_name, record_id, old_value, new_value, performed_at, ip_address}	BIGSERIAL PK — only determinant

INVENTORY_MOVEMENT_LOG (Append-Only Log (Trigger-Managed))

Immutable stock movement ledger. Single BIGSERIAL PK movement_id. batch_id and zone_id are stored as historical snapshots for accuracy.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
movement_id	→	{inventory_id, movement_type, quantity_change, quantity_after, batch_id, zone_id, reference_id, reference_table, performed_by, performed_at}	BIGSERIAL PK — only determinant

4 Normalization Proofs (1NF $\rightarrow$ BCNF), Table-Wise

Each table is analysed stepwise: 1NF $\rightarrow$ 2NF $\rightarrow$ 3NF $\rightarrow$ BCNF. Every claim is accompanied by an explicit reason — not just a verdict. Documented violations are formally classified and then justified as deliberate design trade-offs. Candidate keys are declared first for each table.

SUPPLIER (Strong Entity)

Candidate Keys: {supplier_id}, {supplier_code}, {gstin}, {email}. Prime attributes: supplier_id, supplier_code, gstin, email.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attribute domains contain scalar values. email and phone are real-world multivalued attributes but are simplified to single-valued columns (explicitly documented as a 1NF simplification in the schema). No arrays or nested structures exist. PK = supplier_id. 1NF holds.
2NF	No partial dependency on any composite CK.	✓ PASS — Every CK in SUPPLIER is a single attribute. By definition, a single-attribute CK cannot have a proper subset, so no partial dependency is possible. 2NF is trivially satisfied.
3NF	No non-prime attribute transitively depends on any CK.	✓ PASS — All standard non-prime attributes depend directly on supplier_id with no intermediate non-prime attribute. However: {on_time_delivery_rate, quality_rejection_rate} $\rightarrow$ supplier_rating holds, where both source attributes are non-prime. Strictly, supplier_rating transitively depends on supplier_id via these two non-prime columns. The violation is mitigated in practice because supplier_rating is GENERATED ALWAYS AS STORED. The 3NF violation is thus a formal one with no practical anomaly consequence. Achieved: 3NF (with the documented caveat).
BCNF	Every non-trivial FD's LHS is a superkey.	! NOTE — Checking all non-trivial FDs: PK and alternates $\rightarrow$ {all} ✓(superkeys). Problematic FD: {on_time_delivery_rate, quality_rejection_rate} $\rightarrow$ supplier_rating, where LHS is NOT a superkey — strict BCNF violation. Retained here for KPI performance. Final verdict: 3NF achieved; BCNF violated (documented, justified).

PRODUCT_CATEGORY (Strong Entity — Recursive)

Candidate Keys: {category_id}, {category_name}. Prime attributes: category_id, category_name.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attributes are atomic scalars. Recursive hierarchy is modelled via a nullable self-referential FK. PK = category_id. 1NF holds.

Normal Form	Requirement	Verdict & Detailed Reasoning
2NF	No partial dependency on any composite CK.	✓ PASS — Both CKs are single-attribute. 2NF trivially satisfied.
3NF	No non-prime attribute transitively depends on any CK.	✓ PASS — Non-prime attributes: parent_category_id and description. parent_category_id is a self-referential FK. description is an independent attribute. No chain $A \rightarrow B \rightarrow C$ exists within the table. 3NF holds.
BCNF	Every non-trivial FD's LHS is a superkey.	✓ PASS — Non-trivial FDs: category_id $\rightarrow \{...\}$ and category_name $\rightarrow \{...\}$ are both superkeys. BCNF holds.

PRODUCT (Strong Entity)

Candidate Keys: {product_id}, {sku_code}. Prime attributes: product_id, sku_code.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attributes are scalar atomic types. No multivalued attributes or repeating groups. PK = product_id. 1NF holds.
2NF	No partial dependency on any composite CK.	✓ PASS — Both CKs are single-attribute. 2NF trivially satisfied.
3NF	No non-prime attribute transitively depends on any CK.	✓ PASS — is_perishable and shelf_life_days are conceptually related, but neither functionally determines the other. category_id is a FK reference. 3NF holds.
BCNF	Every non-trivial FD's LHS is a superkey.	✓ PASS — All determinants are superkeys. BCNF holds.

SUPPLIES (Associative Entity — M:N (Supplier $\times$ Product))

Candidate Keys: {supplier_product_id}, {supplier_id, product_id, price_valid_from}. Prime attributes: supplier_product_id, supplier_id, product_id, price_valid_from.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attributes are atomic scalars. PK = supplier_product_id. 1NF holds.
2NF	No partial dependency on any composite CK.	✓ PASS — Checking the composite natural CK. All attributes (unit_price, currency, minimum_order_qty, etc.) depend on the full triple. No partial dependency. 2NF holds.
3NF	No non-prime attribute transitively depends on any CK.	✓ PASS — None of the non-prime attributes determines another non-prime attribute. 3NF holds.
BCNF	Every non-trivial FD's LHS is a superkey.	✓ PASS — Non-trivial FDs are anchored on superkeys. BCNF holds.

WAREHOUSE (Strong Entity)

Candidate Keys: {warehouse_id}, {warehouse_code}. Prime attributes: warehouse_id, warehouse_code.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attributes are scalar atomic values. Physical address is decomposed. 1NF holds.
2NF	No partial dependency on any composite CK.	✓ PASS — Both CKs are single-attribute. 2NF trivially satisfied.
3NF	No non-prime attribute transitively depends on any CK.	✓ PASS — city $\rightarrow$ state could hold in real life, but not as an intra-relation FD enforcing consistency independent of warehouse_id. No other transitive chains. 3NF holds.
BCNF	Every non-trivial FD's LHS is a superkey.	✓ PASS — All determinants are superkeys. BCNF holds.

WAREHOUSE_ZONE (Weak Entity — owner: Warehouse)

Candidate Keys: {zone_id}, {warehouse_id, zone_code}. Prime attributes: zone_id, warehouse_id, zone_code.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attributes are scalar atomic types. 1NF holds.
2NF	No partial dependency on any composite CK.	✓ PASS — zone_type does NOT depend on warehouse_id alone. All attributes require the full composite key. 2NF holds.
3NF	No non-prime attribute transitively depends on any CK.	✓ PASS — temperature_zone and zone_type are correlated conceptually, but correlation is not functional dependency. 3NF holds.
BCNF	Every non-trivial FD's LHS is a superkey.	✓ PASS — All determinants are superkeys. BCNF holds.

BATCH (Strong Entity)

Candidate Keys: {batch_id}, {batch_code}. Prime attributes: batch_id, batch_code.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attributes are scalar atomic values. 1NF holds.
2NF	No partial dependency on any composite CK.	✓ PASS — Both CKs are single-attribute. 2NF trivially satisfied.
3NF	No non-prime attribute transitively depends on any CK.	✓ PASS — supplier_id and product_id are direct properties of the batch. No non-prime attribute determines another. 3NF holds.
BCNF	Every non-trivial FD's LHS is a superkey.	✓ PASS — All determinants are superkeys. BCNF holds.

INVENTORY (Ternary Associative — Product $\times$ Batch $\times$ Warehouse_Zone)

Candidate Keys: {inventory_id}, {batch_id, zone_id}. Prime attributes: inventory_id, batch_id, zone_id.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attributes are scalar atomic types. 1NF holds.
2NF	No partial dependency on any composite CK.	× FAIL — composite natural candidate key: {batch_id, zone_id}. The functional dependency batch_id → product_id exists. Since product_id (non-prime) depends on only a part of the candidate key (batch_id), this is a Partial Dependency. 2NF does not hold.
3NF	No non-prime attribute transitively depends on any CK.	× FAIL — batch_id → product_id holds. Since batch_id is prime, this does not violate 3NF. {quantity_on_hand, quantity_reserved} → quantity_available is a 3NF concern but mitigated by GENERATED ALWAYS enforcement. 3NF holds.
BCNF	Every non-trivial FD's LHS is a superkey.	× FAIL — BCNF violation 1: batch_id → product_id. BCNF violation 2: {quantity_on_hand, quantity_reserved} → quantity_available. Final verdict: BCNF not achieved (both retained with justification).

INVENTORY_ALLOCATION (Associative Entity — SO_Item × Inventory)

Candidate Keys: {allocation_id}. Prime attributes: allocation_id.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — Single-attribute PK. 2NF trivially satisfied.
3NF	No transitive dependency...	✓ PASS — No non-prime determines another non-prime. 3NF holds.
BCNF	LHS is a superkey.	✓ PASS — BCNF holds.

PURCHASE_ORDER (Strong Entity)

Candidate Keys: {po_id}, {po_number}. Prime attributes: po_id, po_number.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — Single-attribute CKs. 2NF holds.
3NF	No transitive dependency...	✓ PASS — grn_status, received_by, etc., functionally depend on po_id, not each other. 3NF holds.
BCNF	LHS is a superkey.	✓ PASS — BCNF holds.

PO_ITEM (Weak Entity — owner: Purchase_Order)

Candidate Keys: {po_item_id}, {po_id, product_id}. Prime attributes: po_item_id, po_id, product_id.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — No attribute depends on po_id or product_id alone. 2NF holds.
3NF	No transitive dependency...	! NOTE — {ordered_quantity, unit_price} → line_total is a transitive dependency among non-prime attributes. Mitigated by GENERATED ALWAYS AS STORED. 3NF is technically violated but practically harmless.
BCNF	LHS is a superkey.	! NOTE — BCNF violation: {ordered_quantity, unit_price} → line_total. Retained as price-at-order-time snapshot.

CUSTOMER (Strong Entity)

Candidate Keys: {customer_id}, {customer_code}, {email}. Prime attributes: customer_id, customer_code, email.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — outstanding_balance is NOT stored. 1NF holds.
2NF	No partial dependency...	✓ PASS — Single-attribute CKs. 2NF holds.
3NF	No transitive dependency...	✓ PASS — credit_limit and is_credit_hold are correlated but not FD. 3NF holds.
BCNF	LHS is a superkey.	✓ PASS — BCNF holds.

SALES_ORDER (Strong Entity)

Candidate Keys: {so_id}, {so_number}. Prime attributes: so_id, so_number.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — Delivery address is decomposed. 1NF holds.
2NF	No partial dependency...	✓ PASS — Single-attribute CKs. 2NF holds.
3NF	No transitive dependency...	✓ PASS — No non-prime → non-prime chain. 3NF holds.
BCNF	LHS is a superkey.	✓ PASS — BCNF holds.

SO_ITEM (Weak Entity — owner: Sales_Order)

Candidate Keys: {so_item_id}, {so_id, product_id}. Prime attributes: so_item_id, so_id, product_id.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — 2NF holds.

Normal Form	Requirement	Verdict & Detailed Reasoning
3NF	No transitive dependency...	× NOTE — Same analysis as PO_ITEM: generated line_total. 3NF technically violated.
BCNF	LHS is a superkey.	× FAIL — BCNF violation retained with justification.

SHIPMENT (Strong Entity)

Candidate Keys: {shipment_id}, {shipment_number}. Prime attributes: shipment_id, shipment_number.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — Single-attribute CKs. 2NF holds.
3NF	No transitive dependency...	× FAIL — driver_name → driver_phone. Transitive dependency among non-prime attributes — genuine 3NF violation. Retained to avoid a 20th table. Final verdict: 3NF violated.
BCNF	LHS is a superkey.	× FAIL — BCNF not achieved.

SHIPMENT_ITEM (Weak Entity — owner: Shipment)

Candidate Keys: {shipment_item_id}, {shipment_id, so_item_id}. Prime attributes: shipment_item_id, shipment_id, so_item_id.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — 2NF holds.
3NF	No transitive dependency...	✓ PASS — Single non-prime attribute. 3NF trivially holds.
BCNF	LHS is a superkey.	✓ PASS — BCNF holds.

RETURN_REQUEST (Strong Entity)

Candidate Keys: {return_id}. Prime attributes: return_id.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — Single-attribute PK. 2NF holds.
3NF	No transitive dependency...	× FAIL — so_id → customer_id holds. return_id → so_id → customer_id. 3NF violated, retained as deliberate denormalization.
BCNF	LHS is a superkey.	× FAIL — BCNF is not achieved.

RETURN_ITEM (Weak Entity — owner: Return_Request)

Candidate Keys: {return_item_id}, {return_id, so_item_id}. Prime attributes: return_item_id, return_id, so_item_id.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — 2NF holds.
3NF	No transitive dependency...	✓ PASS — 3NF holds.
BCNF	LHS is a superkey.	✓ PASS — BCNF holds.

STAFF (Strong Entity)

Candidate Keys: {staff_id}, {employee_code}, {email}. Prime attributes: staff_id, employee_code, email.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — 2NF holds.
3NF	No transitive dependency...	✓ PASS — 3NF holds.
BCNF	LHS is a superkey.	✓ PASS — BCNF holds.

REORDER_ALERT, AUDIT_LOG, INVENTORY_MOVEMENT_LOG

All three system-managed tables satisfy 1NF, 2NF, 3NF, and BCNF. Snapshot columns prevent live transitive dependencies.

Normalization Summary

Table-Wise Statistics

Table	1NF	2NF	3NF	BCNF	Notes
SUPPLIER	✓	✓	✓*	×*	3NF achieved; BCNF violated by generated col. Retained for KPI performance.
PRODUCT_CATEGORY	✓	✓	✓	✓	BCNF achieved. Recursive self-ref FK is an attribute.
PRODUCT	✓	✓	✓	✓	BCNF achieved.
SUPPLIES	✓	✓	✓	✓	BCNF achieved.
WAREHOUSE	✓	✓	✓	✓	BCNF achieved.
WAREHOUSE_ZONE	✓	✓	✓	✓	BCNF achieved.
BATCH	✓	✓	✓	✓	BCNF achieved.
INVENTORY	✓	×	×	×*	3NF achieved. BCNF violated by batch_id → product_id and generated qty_avail. Retained.
INVENTORY_ALLOCATION	✓	✓	✓	✓	BCNF achieved.
PURCHASE_ORDER	✓	✓	✓	✓	BCNF achieved. GRN merge justified.
PO_ITEM	✓	✓	×*	×*	3NF technically violated by generated line_total. BCNF violated. Retained as audit snapshot.
CUSTOMER	✓	✓	✓	✓	BCNF achieved. outstanding_balance NOT stored.
SALES_ORDER	✓	✓	✓	✓	BCNF achieved.
SO_ITEM	✓	✓	×*	×*	Same analysis as PO_ITEM. Retained as audit snapshot.
SHIPMENT	✓	✓	×	×	3NF violated: driver_name → driver_phone. Retained to avoid new table.
SHIPMENT_ITEM	✓	✓	✓	✓	BCNF achieved.
RETURN_REQUEST	✓	✓	×	×	3NF violated: return_id → so_id → customer_id. Deliberate denormalization.
RETURN_ITEM	✓	✓	✓	✓	BCNF achieved.
STAFF	✓	✓	✓	✓	BCNF achieved.
REORDER_ALERT	✓	✓	✓	✓	BCNF achieved.
AUDIT_LOG	✓	✓	✓	✓	BCNF achieved.
INVENTORY_MOVE-MENT_LOG	✓	✓	✓	✓	BCNF achieved.

* = violation documented and retained with explicit design trade-off justification.

Aggregate Statistics

Category	Count	Tables
BCNF achieved (fully normalized)	16 of 22 (incl. 3 DDL Log Tables)	Product_Category, Product, Supplies, Warehouse, Warehouse_Zone, Batch, Inventory_Allocation, Purchase_Order, Customer, Sales_Order, Shipment_Item, Return_Item, Staff, Reorder_Alert, Audit_Log, Inventory_Movement_Log
3NF achieved; BCNF violated (generated col, retained)	1	Supplier
3NF technically violated by generated col (retained)	2	PO_Item, SO_Item
3NF violated, genuine intra-table transitive FD	1	Shipment
3NF violated, deliberate cross-table denormalization	1	Return_Request
2NF violated	1	Inventory

All violations are explicitly documented, formally classified, and retained with justified design trade-offs.

5 Notes

This final document has all the submissions required in PDF format. The other files to be attached, which are stated below, have been compressed into a single zip file.

- DDL scripts - `.txt` file
- INSERT scripts - `.txt` file
- Queries (Query Text) and their Solutions (SQL SELECT QUERIES) - `.txt` file